

Министерство образования и науки Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
ИТМО»

ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ
Направление подготовки: 11.03.02 Инфокоммуникационные технологии
и системы связи

Лабораторная работа №6

Выполнил:
Бацанова Екатерина Александровна
Группа К3323
Проверил:
Марченко Елена Вадимовна

Санкт-Петербург
2025

СОДЕРЖАНИЕ

Стр.

Цель	3
Ход работы	4
1 Задание 1.....	4
2 Задание 2.....	6
3 Задание 3.....	13
4 Задание 4.....	20
Вывод.....	27

ЦЕЛЬ

Цель лабораторной работы – овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Для выполнения работы можно выделить следующие цели:

1. Реализовать клиента и сервер, которые обмениваются сообщениями ("Hello, server "Hello, client");
2. Реализовать клиент-серверное приложение, где клиент отправляет серверу запрос на выполнение математической операции, передавая параметры через клавиатуру. Сервер обрабатывает запрос и возвращает результат клиенту;
3. Реализовать сервер, который по запросу клиента отправляет HTML-страницу, загруженную из файла index.html. Клиент должен подключиться к серверу и получить HTML-страницу;
4. Реализуйте многопользовательский чат. Сервер должен поддерживать несколько клиентов, которые могут отправлять сообщения, видимые всем участникам чата. Также, при выходе пользователя из чата, остальные пользователи должны получать уведомление о его выходе.

ХОД РАБОТЫ

1 Задание 1

В данном задании было необходимо реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение "Hello, server". Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение "Hello, client". Сообщение должно отобразиться у клиента.

Сначала реализуем сервер (рис. 1). Здесь используется `socket.socket()` для создания TCP-сокета, с помощью `bind()` сервер привязывается к заданному IP-адресу и порту, а `listen()` ставит сервер в режим ожидания соединений. Метод `accept()` принимает входящие подключения от клиентов. Сервер получает сообщение от клиента через `recv()` и отправляет ответ через `sendall()`. После ответа сервер продолжает ожидать новых подключений. Если сервер получает сигнал о завершении (Ctrl+C), программа завершает работу.

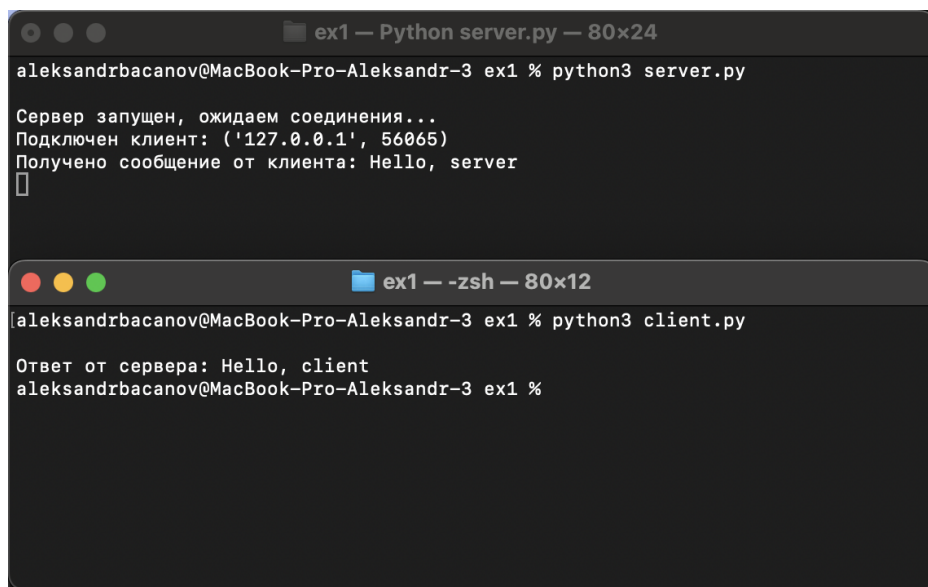
```
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 65432
5  try:
6      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7          s.bind((HOST, PORT))
8          s.listen()
9
10         print("Сервер запущен, ожидаем соединения...")
11
12         while True:
13             conn, addr = s.accept()
14             with conn:
15                 print(f"Подключен клиент: {addr}")
16                 data = conn.recv(1024)
17                 print(f"Получено сообщение от клиента: {data.decode()}")
18
19                 response = "Hello, client"
20                 conn.sendall(response.encode())
21
22 except KeyboardInterrupt:
23     print("\nСервер остановлен пользователем (Ctrl+C)")
```

Рисунок 1 — Код файла *server.py*.

Также реализуем клиента. Код на рис. 2 создает клиентский сокет, подключается к серверу по адресу 127.0.0.1 и порту 65432 (на которых слушает сервер), отправляет сообщение "Hello, server" затем получает ответ от сервера и выводит его на экран.

```
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 65432
5
6  with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7      s.connect((HOST, PORT))
8
9      message = "Hello, server"
10     s.sendall(message.encode())
11
12     data = s.recv(1024)
13     print(f"Ответ от сервера: {data.decode()}")
```

Рисунок 2 — Код файла *client.py*.



The image shows two terminal windows. The top window, titled "ex1 — Python server.py — 80x24", shows the execution of `python3 server.py`. The output is: "Сервер запущен, ожидаем соединения...", "Подключен клиент: ('127.0.0.1', 56065)", and "Получено сообщение от клиента: Hello, server". The bottom window, titled "ex1 — -zsh — 80x12", shows the execution of `python3 client.py`, which outputs "Ответ от сервера: Hello, client".

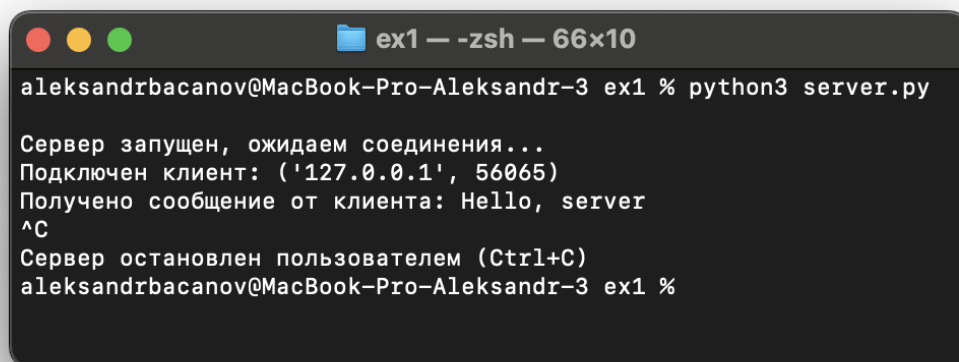
```
ex1 — Python server.py — 80x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex1 % python3 server.py

Сервер запущен, ожидаем соединения...
Подключен клиент: ('127.0.0.1', 56065)
Получено сообщение от клиента: Hello, server
█

ex1 — -zsh — 80x12
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex1 % python3 client.py

Ответ от сервера: Hello, client
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex1 %
```

Рисунок 3 — Пример взаимодействия клиента и сервера.



```
ex1 — -zsh — 66x10
aleksandrbacanov@MacBook-Pro-Aleksandr-3 ex1 % python3 server.py
Сервер запущен, ожидаем соединения...
Подключен клиент: ('127.0.0.1', 56065)
Получено сообщение от клиента: Hello, server
^C
Сервер остановлен пользователем (Ctrl+C)
aleksandrbacanov@MacBook-Pro-Aleksandr-3 ex1 %
```

Рисунок 4 — Пример завершения работы сервера.

2 Задание 2

Во втором задании было необходимо реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту. Варианты:

- а. Теорема Пифагора;
- б. Решение квадратного уравнения;
- с. Поиск площади трапеции;
- д. Поиск площади параллелограмма.

Я реализовала все предложенные варианты.

Реализуем сервер (рис. 5, рис. 6, рис. 7). Этот код реализует многопользовательский сервер, который принимает запросы от клиентов и выполняет математические вычисления, перечисленные выше.

Сервер слушает на порту 65432 и принимает подключения клиентов, создавая для каждого клиента отдельный поток. Клиенты отправляют запросы

с номерами задач и параметрами. Сервер выполняет вычисления в зависимости от запроса и отправляет результат вычислений обратно клиенту.

Если клиент отправляет "exit" соединение с ним закрывается. Если ввод некорректен, отправляется ошибка. Ввод команды shutdown завершает работу сервера, отключает всех клиентов и завершает все потоки. Подобно прошлому коду, сервер работает в многопоточном режиме, обрабатывая несколько клиентов одновременно.

```
1  import socket
2  import threading
3
4  HOST = '127.0.0.1'
5  PORT = 65432
6  client_connections = []
7  client_threads = []
8  server_running = True
9
10 def pythagorean_theorem(a, b): 1 usage
11     return (a**2 + b**2) ** 0.5
12
13 def solve_quadratic(a, b, c): 1 usage
14     d = b ** 2 - 4 * a * c
15     if d > 0:
16         x1 = (-b + d ** 0.5) / (2 * a)
17         x2 = (-b - d ** 0.5) / (2 * a)
18         return f"Корни уравнения: x1 = {x1}, x2 = {x2}"
19     elif d == 0:
20         x = -b / (2 * a)
21         return f"Один корень: x = {x}"
22     else:
23         return "Корней нет"
24
25 def trapezoid_area(a, b, h): 1 usage
26     return 0.5 * (a + b) * h
27
28 def parallelogram_area(base, height): 1 usage
29     return base * height
30
31 def validate_positive(value): 5 usages
32     return value > 0
```

Рисунок 5 — Часть 1: код файла *server.py*, реализация математических операций и валидации.

```

34 def handle_client(conn, addr): 1 usage
35     print(f"Подключен клиент: {addr}")
36     client_connections.append(conn)
37     try:
38         with conn:
39             while True:
40                 try:
41                     data = conn.recv(1024)
42                     request = data.decode()
43                     if request == 'exit':
44                         print(f"Клиент {addr} отправил 'exit'.")
45                         break
46                     params = conn.recv(1024).decode().split()
47                     try:
48                         if request == '1':
49                             a, b = map(float, params)
50                             if validate_positive(a) and validate_positive(b):
51                                 result = pythagorean_theorem(a, b)
52                             else:
53                                 result = "Ошибка: параметры должны быть положительными."
54                         elif request == '2':
55                             a, b, c = map(float, params)
56                             result = solve_quadratic(a, b, c)
57                         elif request == '3':
58                             a, b, h = map(float, params)
59                             if all(map(validate_positive, [a, b, h])):
60                                 result = trapezoid_area(a, b, h)
61                             else:
62                                 result = "Ошибка: параметры должны быть положительными."
63                         elif request == '4':
64                             base, height = map(float, params)
65                             if validate_positive(base) and validate_positive(height):
66                                 result = parallelogram_area(base, height)
67                             else:
68                                 result = "Ошибка: параметры должны быть положительными."
69                         else:
70                             result = "Неизвестная команда."
71                     except (ValueError, IndexError):
72                         result = "Ошибка: некорректный ввод."
73                     conn.sendall(str(result).encode())
74                 except (ConnectionResetError, BrokenPipeError):
75                     print(f"Клиент {addr} разорвал соединение.")
76                     break
77                 except OSError:
78                     # сокет закрыт сервером — завершение
79                     break
80             except Exception as e:
81                 print(f"Ошибка в потоке клиента {addr}: {e}")
82             finally:
83                 if conn in client_connections:
84                     client_connections.remove(conn)
85                 print(f"Клиент {addr} отключен.")

```

Рисунок 6 — Часть 2: код файла *server.py*, обработка клиента на сервере.


```

87 def server_main(): 1 usage
88     global server_running
89     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
90         s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
91         s.bind((HOST, PORT))
92         s.listen()
93         s.settimeout(1.0)
94         print("Сервер запущен, ожидаем соединений...")
95
96         while server_running:
97             try:
98                 conn, addr = s.accept()
99                 thread = threading.Thread(target=handle_client, args=(conn, addr))
100                 thread.start()
101                 client_threads.append(thread)
102             except socket.timeout:
103                 continue
104             except Exception as e:
105                 print(f"Ошибка сервера: {e}")
106                 break
107
108         print("Сервер завершает работу, отключаем клиентов...")
109
110         for client in client_connections[:]:
111             try:
112                 client.shutdown(socket.SHUT_RDWR)
113                 client.close()
114             except Exception as e:
115                 print(f"Ошибка при отключении клиента: {e}")
116
117         for thread in client_threads:
118             thread.join()
119
120         print("Сервер завершён.")
121
122     server_thread = threading.Thread(target=server_main)
123     server_thread.start()
124
125     while True:
126         command = input("Введите 'shutdown' для остановки сервера: ").strip().lower()
127         if command == 'shutdown':
128             server_running = False
129             server_thread.join()
130             break

```

Рисунок 7 — Часть 3: код файла *server.py*, основная логика сервера.

Теперь реализуем клиента (рис. 8, рис. 9). Этот код реализует клиентскую часть, которая подключается к серверу, отправляет запросы на выполнение математических операций, а затем получает и выводит результаты. Клиент может выбрать одну из операций, вводит необходимые данные, и отправляет их на сервер для обработки. Если выбор некорректен, пользователю предлагается попробовать снова. В случае ошибок подключения или получения данных выводятся соответствующие сообщения.

```

1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 65432
5
6  def request_data_positive(prompt): 7 usages
7      while True:
8          try:
9              value = float(input(prompt))
10             if value <= 0:
11                 print("Ошибка: число должно быть положительным.")
12                 continue
13             return value
14         except ValueError:
15             print("Ошибка: введите число, пожалуйста.")
16
17  def request_data_any(prompt): 3 usages
18      while True:
19          try:
20              return float(input(prompt))
21         except ValueError:
22             print("Ошибка: введите число, пожалуйста.")
23
24  try:
25      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
26          s.connect((HOST, PORT))
27
28          while True:
29              print("\nВыберите операцию:")
30              print("1      | Теорема Пифагора")
31              print("2      | Решение квадратного уравнения")
32              print("3      | Площадь трапеции")
33              print("4      | Площадь параллелограмма")
34              print("'exit' | Выход")
35
36              choice = input("Введите номер операции или 'exit' для выхода: ").strip()
37
38              try:
39                  s.sendall(choice.encode())
40             except Exception:
41                 print("Соединение с сервером прервано. Завершение работы.")
42                 break
43
44             if choice == 'exit':
45                 print("Отключение от сервера...")
46                 break

```

Рисунок 8 — Часть 1: код файла *client.py*, выбор математической операции и отправка данных на сервер.

```

25         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
48             if choice == '1':
49                 a = request_data_positive("Введите катет a: ")
50                 b = request_data_positive("Введите катет b: ")
51                 params = f"{a} {b}"
52             elif choice == '2':
53                 a = request_data_any("Введите коэффициент a: ")
54                 b = request_data_any("Введите коэффициент b: ")
55                 c = request_data_any("Введите коэффициент c: ")
56                 params = f"{a} {b} {c}"
57             elif choice == '3':
58                 a = request_data_positive("Введите основание a: ")
59                 b = request_data_positive("Введите основание b: ")
60                 h = request_data_positive("Введите высоту h: ")
61                 params = f"{a} {b} {h}"
62             elif choice == '4':
63                 base = request_data_positive("Введите основание: ")
64                 height = request_data_positive("Введите высоту: ")
65                 params = f"{base} {height}"
66             else:
67                 print("Некорректный выбор, попробуйте снова.")
68                 continue
69
70             try:
71                 s.sendall(params.encode())
72             except Exception:
73                 print("Ошибка при отправке данных. Сервер недоступен.")
74                 break
75
76             try:
77                 data = s.recv(1024)
78                 if not data:
79                     print("Сервер закрыл соединение.")
80                     break
81
82                 response = data.decode()
83
84                 print(f"Результат: {response}")
85             except Exception:
86                 print("Ошибка при получении ответа от сервера.")
87                 break
88
89         except ConnectionRefusedError:
90             print("Не удалось подключиться к серверу. Убедитесь, что он запущен.")

```

Рисунок 9 — Часть 2: код файла *client.py*, обработка ввода данных для математических операций и обмен информацией с сервером.

```
ex2 — -zsh — 80x58
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 % python3 client.py

Выберите операцию:
1      | Теорема Пифагора
2      | Решение квадратного уравнения
3      | Площадь трапеции
4      | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: 1
Введите катет a: 1
Введите катет b: 1
Результат: 1.4142135623730951

Выберите операцию:
1      | Теорема Пифагора
2      | Решение квадратного уравнения
3      | Площадь трапеции
4      | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: 2
Введите коэффициент a: -1
Введите коэффициент b: 2
Введите коэффициент c: -1
Результат: Один корень: x = 1.0

Выберите операцию:
1      | Теорема Пифагора
2      | Решение квадратного уравнения
3      | Площадь трапеции
4      | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: 3
Введите основание a: 1
Введите основание b: 2
Введите высоту h: 3
Результат: 4.5

Выберите операцию:
1      | Теорема Пифагора
2      | Решение квадратного уравнения
3      | Площадь трапеции
4      | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: 4
Введите основание: 1
Введите высоту: 2
Результат: 2.0

Выберите операцию:
1      | Теорема Пифагора
2      | Решение квадратного уравнения
3      | Площадь трапеции
4      | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: exit
Отключение от сервера...
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 %
```

Рисунок 10 — Пример работы со стороны клиента.

```
ex2 — Python server.py — 80x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 % python3 server.py

Введите 'shutdown' для остановки сервера: Сервер запущен, ожидаем соединений...
Подключен клиент: ('127.0.0.1', 58261)
Клиент ('127.0.0.1', 58261) отправил 'exit'.
Клиент ('127.0.0.1', 58261) отключен.
```

Рисунок 11 — Пример работы со стороны сервера.

```
ex2 — -zsh — 80x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 % python3 server.py

Введите 'shutdown' для остановки сервера: Сервер запущен, ожидаем соединений...
Подключен клиент: ('127.0.0.1', 59980)
shutdown
Сервер завершает работу, отключаем клиентов...
Клиент ('127.0.0.1', 59980) отключен.
Сервер завершён.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 %

ex2 — -zsh — 80x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 % python3 client.py

Выберите операцию:
1 | Теорема Пифагора
2 | Решение квадратного уравнения
3 | Площадь трапеции
4 | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: sdc
Некорректный выбор, попробуйте снова.

Выберите операцию:
1 | Теорема Пифагора
2 | Решение квадратного уравнения
3 | Площадь трапеции
4 | Площадь параллелограмма
'exit' | Выход
Введите номер операции или 'exit' для выхода: sdc
Соединение с сервером прервано. Завершение работы.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex2 %
```

Рисунок 12 — Пример работы отключения сервера.

3 Задание 3

В этом задании было необходимо реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащеее html-страницу, которую сервер подгружает из файла index.html.

Реализуем сервер (рис. 13). Этот код реализует простой http-сервер. Сервер создаёт сокет и привязывает его к адресу 127.0.0.1 и порту 65432. Он начинает прослушивание входящих подключений. Когда клиент подключается, сервер получает запрос и извлекает путь запрашиваемого ресурса (например, */index.html*). Если путь совпадает с */index.html* или */style.css*, сервер пытается открыть и прочитать соответствующий файл. Если файл существует, сервер возвращает его содержимое с HTTP-статусом 200 (ОК). Если файл не найден, возвращается ошибка 404. Сервер формирует ответ в формате http, включая заголовки и содержимое, и отправляет его клиенту.

Если мы нажимаем Ctrl+C, сервер завершает свою работу.

CSS файл обрабатывается отдельно для указания правильного типа контента в http-заголовке. Для html используется *text/html*, а для css — *text/css*. Это позволяет браузеру корректно интерпретировать и применять стили к веб-странице. Без правильного типа контента браузер может неправильно обработать файл, рассматривая его как обычный текст.

```

1  import socket
2  import os
3
4  HOST = '127.0.0.1'
5  PORT = 65432
6
7  def read_file(filename, content_type): 2 usages
8      if not os.path.exists(filename):
9          return f"""HTTP/1.1 404 Not Found\r
10 Content-Type: text/html\r
11 \r
12 <h1>Файл {filename} не найден</h1>
13 """.encode()
14
15     with open(filename, "r", encoding="utf-8") as f:
16         content = f.read()
17     return f"""HTTP/1.1 200 OK\r
18 Content-Type: {content_type}; charset=UTF-8\r
19 \r
20 {content}
21 """.encode()
22
23 try:
24     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
25         s.bind((HOST, PORT))
26         s.listen(1)
27         print(f"Сервер запущен на {HOST}:{PORT}...")
28
29         while True:
30             conn, addr = s.accept()
31             with conn:
32                 print(f"Подключен клиент: {addr}")
33                 request = conn.recv(1024).decode()
34                 print(f"Запрос от клиента:\n{request}")
35                 if not request:
36                     continue
37                 request_line = request.splitlines()[0]
38                 path = request_line.split(" ")[1]
39
40                 if path == "/" or path == "/index.html":
41                     response = read_file(filename="index.html", content_type="text/html")
42                 elif path == "/style.css":
43                     response = read_file(filename="style.css", content_type="text/css")
44                 else:
45                     response = f"""HTTP/1.1 404 Not Found\r
46 Content-Type: text/html\r
47 \r
48 <h1>404 - Не найдено</h1>""".encode()
49
50                 conn.sendall(response)
51
52 except KeyboardInterrupt:
53     print("\nСервер остановлен пользователем (Ctrl+C)")

```

Рисунок 13 — Код файла *server.py*.

```
1      <!DOCTYPE html>
2      <html lang="ru">
3      <head>
4          <meta charset="UTF-8">
5          <meta name="viewport" content="width=device-width, initial-scale=1.0">
6          <title>Добро пожаловать</title>
7          <link rel="stylesheet" href="style.css">
8      </head>
9      <body>
10         <div class="container">
11             <h1>Добро пожаловать на мой сервер!</h1>
12             <p>Это простая страница, обслуживаемая сервером на Python.</p>
13             <div class="footer">
14                 &copy; 2025 Мой мини-сервер
15             </div>
16         </div>
17     </body>
18 </html>
```

Рисунок 14 — Код файла *index.html*.


```
1  body {
2      margin: 0;
3      padding: 0;
4      font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
5      background: linear-gradient(to right, #83a4d4, #b6fbff);
6      display: flex;
7      justify-content: center;
8      align-items: center;
9      height: 100vh;
10 }
11
12 .container {
13     background-color: #ffffffcc;
14     padding: 40px;
15     border-radius: 15px;
16     box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
17     text-align: center;
18     max-width: 500px;
19 }
20
21 h1 {
22     color: #2c3e50;
23     margin-bottom: 20px;
24 }
25
26 p {
27     color: #34495e;
28     font-size: 18px;
29 }
30
31 .footer {
32     margin-top: 30px;
33     font-size: 14px;
34     color: #7f8c8d;
35 }
```

Рисунок 15 — Код файла *style.css*.

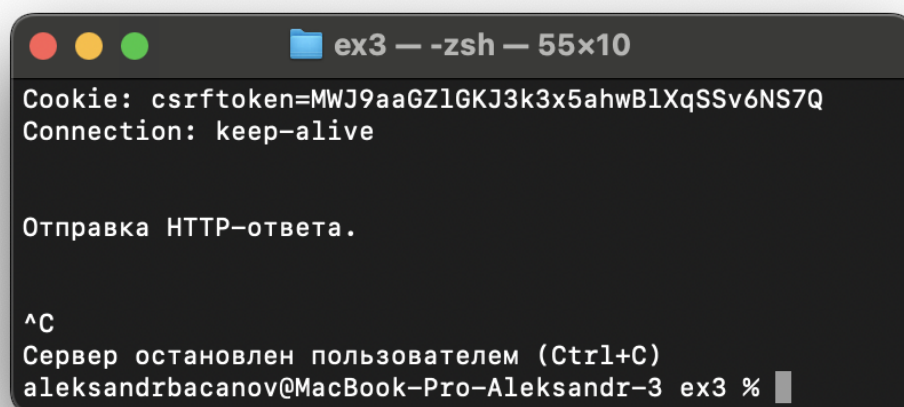
```
ex3 — Python server.py — 80x42
aleksandrbacanov@MacBook-Pro-Aleksandr-3 ex3 % python3 server.py

Сервер запущен и слушает 127.0.0.1:65432...
Подключен клиент: ('127.0.0.1', 58588)
Запрос от клиента:
GET / HTTP/1.1
Host: 127.0.0.1:65432
Sec-Fetch-Dest: document
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/18.3.1 Safari/605.1.15
Upgrade-Insecure-Requests: 1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Accept-Language: ru
Priority: u=0, i
Accept-Encoding: gzip, deflate
Cookie: csrftoken=MWJ9aaGZlGKJ3k3x5ahwB1XqSSv6NS7Q
Connection: keep-alive

Отправка HTTP-ответа.
Подключен клиент: ('127.0.0.1', 58591)
Запрос от клиента:
GET / HTTP/1.1
Host: 127.0.0.1:65432
Sec-Fetch-Dest: document
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/18.3.1 Safari/605.1.15
Upgrade-Insecure-Requests: 1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Accept-Language: ru
Priority: u=0, i
Accept-Encoding: gzip, deflate
Cookie: csrftoken=MWJ9aaGZlGKJ3k3x5ahwB1XqSSv6NS7Q
Connection: keep-alive

Отправка HTTP-ответа.
```

Рисунок 16 — Пример работы со стороны сервера.

A terminal window titled "ex3 — -zsh — 55x10" with a dark background. It displays the output of an HTTP request, including a CSRF token and a keep-alive connection. The text "Отправка HTTP-ответа." is shown. The user presses Ctrl+C, indicated by "^C", and the server is stopped by the user. The prompt shows the user is "aleksandrbacanov@MacBook-Pro-Aleksandr-3" in the "ex3" directory.

```
Cookie: csrftoken=MWJ9aaGZlGKJ3k3x5ahwB1XqSSv6NS7Q
Connection: keep-alive

Отправка HTTP-ответа.

^C
Сервер остановлен пользователем (Ctrl+C)
aleksandrbacanov@MacBook-Pro-Aleksandr-3 ex3 %
```

Рисунок 17 — Пример остановки сервера.

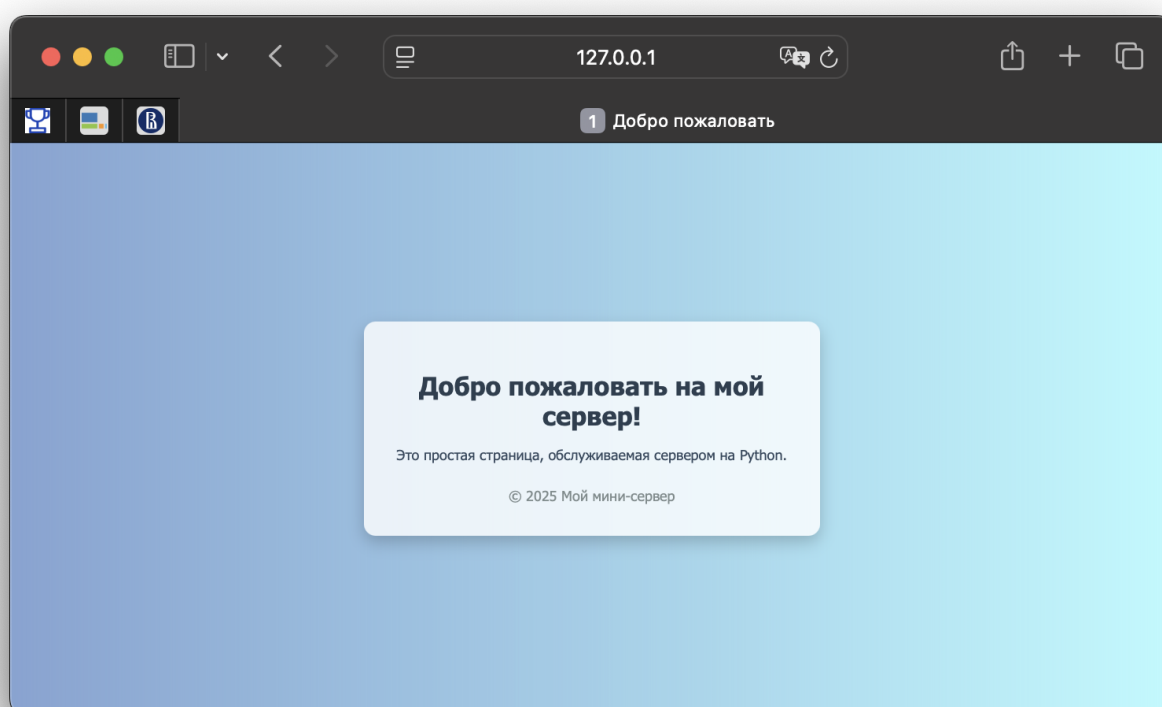


Рисунок 18 — Пример работы со стороны клиента.

4 Задание 4

В этом задании было необходимо реализовать многопользовательский чат.

Начнем с сервера (рис. 19, рис. 20, рис. 21). Этот код реализует многопользовательский чат-сервер с использованием сокетов и потоков. Сервер запускается на локальном IP-адресе 127.0.0.1 и порту 65432, принимает подключения от клиентов и позволяет им общаться друг с другом. При подключении каждый клиент должен ввести уникальный никнейм, который сохраняется в списке `nicknames`, а его соединение — в списке `clients`. Сервер обрабатывает каждого клиента в отдельном потоке, что позволяет обслуживать сразу несколько пользователей.

Клиенты могут отправлять обычные сообщения, которые рассылаются всем остальным пользователям (`broadcast`), или личные сообщения, начиная сообщение с символа `@`, за которым следует никнейм получателя. Сервер также обрабатывает команду `exit` от клиента для выхода из чата, удаляя его из списков и уведомляя остальных. Команда `shutdown`, введенная в консоли сервера, завершает работу: всем клиентам отправляется сообщение о закрытии, соединения закрываются, и сервер останавливается.

Код также обрабатывает ошибки подключения, повторные никнеймы, неактивность клиентов и завершение работы по `Ctrl+C`.

```

1  import socket
2  import threading
3
4  HOST = '127.0.0.1'
5  PORT = 65432
6
7  clients = []
8  nicknames = []
9  server_running = True
10
11 def broadcast(message, client): 3 usages
12     for c in clients:
13         if c != client:
14             try:
15                 c.send(message)
16             except:
17                 clients.remove(c)
18                 nicknames.remove(nicknames[clients.index(c)])
19
20 def send_private_message(sender, nickname, message): 1 usage
21     try:
22         index = nicknames.index(nickname)
23         client = clients[index]
24         client.send(f"Личное сообщение от {sender}: {message}".encode())
25     except ValueError:
26         return None
27     return True
28
29 def handle(client, nickname): 1 usage
30     while True:
31         try:
32             message = client.recv(1024).decode()
33
34             if message == 'exit':
35                 print(f"{nickname} покинул чат.")
36                 client.send('Вы покинули чат.'.encode())
37                 broadcast(f"Пользователь {nickname} вышел из чата.".encode(), client)
38                 clients.remove(client)
39                 nicknames.remove(nickname)
40                 client.close()
41                 break

```

Рисунок 19 — Часть 1: код файла *server.py*, обработка сообщений и управление клиентами чата.

```

29 def handle(client, nickname): 1 usage
42
43     if message.startswith('@'):
44         parts = message.split(' ', 1)
45         if len(parts) == 2:
46             recipient_nickname = parts[0][1:]
47             private_message = parts[1]
48             if send_private_message(nickname, recipient_nickname, private_message) is None:
49                 client.send(f"Ошибка: Пользователь с никнеймом {recipient_nickname} не найден.".encode())
50             else:
51                 client.send("Ошибка: Неверный формат личного сообщения.".encode())
52         else:
53             broadcast(f"{nickname}: {message}".encode(), client)
54     except:
55         print(f"{nickname} покинул чат (неактивность или ошибка соединения).")
56         clients.remove(client)
57         nicknames.remove(nickname)
58         client.close()
59         break
60
61 def receive(): 1 usage
62     while server_running:
63         try:
64             client, address = server.accept()
65             print(f"Подключен: {str(address)}")
66
67             while True:
68                 client.send("Введите свой никнейм: ".encode())
69                 nickname = client.recv(1024).decode()
70
71                 if nickname in nicknames:
72                     client.send("Ошибка: Никнейм уже занят, выберите другой.".encode())
73                     continue
74                 else:
75                     break
76
77             clients.append(client)
78             nicknames.append(nickname)
79
80             broadcast(f'{nickname} присоединился к чату!'.encode(), client)
81             client.send('Вы подключены к чату!'.encode())
82
83             thread = threading.Thread(target=handle, args=(client, nickname))
84             thread.start()
85
86         except Exception as e:
87             if server_running:
88                 print(f"Ошибка при принятии нового подключения: {e}")

```

Рисунок 20 — Часть 2: код файла *server.py*, обработка личных и общих сообщений + подключение новых клиентов.

```

90 def stop_server(): 1 usage
91     global server_running
92     server_running = False
93     print("Завершаем работу сервера...")
94
95     for client in clients[:]:
96         try:
97             print(f"Отправляем сообщение клиенту {client}")
98             client.send("Сервер завершил свою работу. Закрытие соединения...".encode())
99         except Exception as e:
100             print(f"Ошибка при отправке сообщения клиенту {client}: {e}")
101             pass
102
103     # Теперь закрываем соединения с клиентами
104     for client in clients[:]:
105         try:
106             client.close()
107             clients.remove(client)
108         except Exception as e:
109             pass
110
111     server.close()
112     print("Сервер завершил свою работу.")
113
114     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
115     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
116     server.bind((HOST, PORT))
117     server.listen()
118
119     print(f"Сервер запущен на {HOST}:{PORT}")
120
121     thread = threading.Thread(target=receive)
122     thread.daemon = True
123     thread.start()
124
125     while server_running:
126         command = input("Введите 'shutdown' для завершения работы сервера: ")
127         if command == 'shutdown':
128             stop_server()
129             break

```

Рисунок 21 — Часть 3: код файла *server.py*, остановка сервера и завершение соединений.

Теперь реализуем клиентскую часть (рис. 22). Этот код реализует клиент для чата, который подключается к серверу, отправляет свой никнейм и участвует в переписке. Клиент работает в двух потоках: один принимает сообщения от сервера и выводит их в консоль, другой — считывает ввод пользователя и отправляет его на сервер. Если пользователь вводит `exit` или сервер завершает работу, соединение закрывается. Также предусмотрена обработка ошибок, таких как потеря связи или отказ в подключении.


```

1  import socket
2  import threading
3  import sys
4
5  HOST = '127.0.0.1'
6  PORT = 65432
7
8  def receive_messages(client): 1 usage
9      while True:
10         try:
11             message = client.recv(1024).decode()
12             if not message:
13                 continue
14
15             if message.startswith("Сервер завершает работу"):
16                 print("Сервер завершил свою работу. Вы отключены.")
17                 client.close()
18                 break
19
20             print(message)
21         except:
22             print("Вы потеряли соединение с сервером.")
23             client.close()
24             break
25     sys.exit(0)
26
27 def send_messages(client): 1 usage
28     while True:
29         try:
30             message = input()
31             if message.lower() == "exit":
32                 client.send("exit".encode())
33                 print("Вы покинули чат.")
34                 client.close()
35                 break
36             client.send(message.encode())
37         except:
38             break
39     try:
40         client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
41         client.connect((HOST, PORT))
42         nickname = input("Введите свой никнейм: ")
43         client.send(nickname.encode())
44         receive_thread = threading.Thread(target=receive_messages, args=(client,))
45         receive_thread.start()
46         send_thread = threading.Thread(target=send_messages, args=(client,))
47         send_thread.start()
48
49     except ConnectionRefusedError:
50         print("Не удалось подключиться к серверу.")
51     except KeyboardInterrupt:
52         print("Клиент завершил работу.")

```

Рисунок 22 — Код файла *client.py*.

```
ex4 -- zsh -- 71x25
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 % python3 server.py
Сервер запущен на 127.0.0.1:65432
Введите команду для завершения работы сервера или 'shutdown' для завершения: Подключен: ('127.0.0.1', 59168)
Подключен: ('127.0.0.1', 59169)
Подключен: ('127.0.0.1', 59170)
gini покинул чат.
Подключен: ('127.0.0.1', 59171)
shutdown
Завершаем работу сервера...
Отправляем сообщение клиенту <socket.socket fd=4, family=2, type=1, proto=0, laddr=('127.0.0.1', 65432), raddr=('127.0.0.1', 59168)>
Отправляем сообщение клиенту <socket.socket fd=6, family=2, type=1, proto=0, laddr=('127.0.0.1', 65432), raddr=('127.0.0.1', 59170)>
Отправляем сообщение клиенту <socket.socket fd=5, family=2, type=1, proto=0, laddr=('127.0.0.1', 65432), raddr=('127.0.0.1', 59171)>
kathy покинул чат (неактивность или ошибка соединения).
qwerty покинул чат (неактивность или ошибка соединения).
gini покинул чат (неактивность или ошибка соединения).
Сервер завершил свою работу.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 %

ex4 -- zsh -- 80x25
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 % python3 client.py
Введите свой никнейм: kathy
Введите свой никнейм: Вы подключены к чату!
gini присоединился к чату!
qwerty присоединился к чату!
Личное сообщение от qwerty: hi
hello everyone
Пользователь gini вышел из чата.
gini присоединился к чату!
Сервер завершил свою работу. Закрытие соединения...
с
Вы потеряли соединение с сервером.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 %

ex4 -- zsh -- 71x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 % python3 client.py
Введите свой никнейм: kathy
Введите свой никнейм: Ошибка: Никнейм уже занят, выберите другой.Введите свой никнейм:
gini
Вы подключены к чату!
qwerty присоединился к чату!
kathy: hello everyone
exit
Вы покинули чат.
Вы потеряли соединение с сервером.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 % python3 client.py
Введите свой никнейм: gini
Введите свой никнейм: Вы подключены к чату!
Сервер завершил свою работу. Закрытие соединения...
с
Вы потеряли соединение с сервером.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 %

ex4 -- zsh -- 80x24
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 % python3 client.py
Введите свой никнейм: qwerty
Введите свой никнейм: Вы подключены к чату!
@kathy hi
kathy: hello everyone
Пользователь gini вышел из чата.
gini присоединился к чату!
Сервер завершил свою работу. Закрытие соединения...
с
Вы потеряли соединение с сервером.
aleksandrbaranov@MacBook-Pro-Aleksandr-3 ex4 %
```

Рисунок 23 — Пример работы чата.

ВЫВОД

В ходе работы были реализованы ключевые аспекты создания серверных приложений с использованием сокетов, что позволило освоить основы сетевого программирования и взаимодействия между клиентом и сервером. Решены задачи, включающие обмен сообщениями, обработку математических запросов, работу с http-протоколом и создание многопользовательского чата.